

Construção de Compiladores

Especificação Léxica e Analisador Léxico

Cauã Pereira Neres - 12021BCC005

2. Parte 1 — Especificação léxica da linguagem

Token	Notação (regex/EBNF)	Exemplos válidos
Identificador	<code>[A-Za-z][A-Za-z0-9_]*</code>	soma, Soma, s1, S1, Soma_1, SoMa, sOmA
Palavra reservada	mesmo padrão do identificador + tabela de busca	int, if, while.
String	<code>"[^\"\\n]*"</code>	“churrasco”, “Avenida”, “oVo”, “Tudo bem”, “Como vai ?”, “abc123”.
Operador	<code>OPERADOR = { +, -, *, /, %, =, ==, <, <=, >, >=, !=, &&, }</code>	<code>=, ==, <=, +, &&.</code>
Literal numérico	<code>[0-9]+ (\. [0-9]+)?</code>	10, 3.14, 0
Delimitador	<code>[\(\)]</code>	(,)

1)Qual é o alfabeto de entrada (que caracteres são válidos em algum ponto de algum token)?

Letras: a-z, A-Z

Dígitos: 0-9

Símbolos: + - * / % > < & () { } ; . , | _ ! = “ ”

Espaços: space, tab, newline

2)A linguagem é case-sensitive? Palavras reservadas e identificadores seguem a mesma regra de maiúsculas/minúsculas?

R: A linguagem é case-sensitive, com os identificadores e palavras reservadas seguindo a regra de maiúsculas/minúsculas.

3)Como são delimitados espaços em branco e comentários (linha e/ou bloco)? Comentários de bloco podem ser aninhados?

R: A linguagem vai possuir comentários de linhas, iniciados por // e terminados com quebra de linha, e comentários de blocos, iniciados com /* e terminados por */. Comentários de bloco não podem ser aninhados.

4)Para operadores com forma composta (ex.: = vs. ==), qual é a regra de desambiguação — "maximal munch": o scanner sempre tenta consumir o prefixo válido mais longo antes de decidir o token.

R: Sim, vai ser usado o maximal munch (casamento mais longo).

5)Quais são exatamente as palavras reservadas da linguagem (lista fechada, não apenas exemplos)?

Lista de palavras reservadas: int, double, bool, char, string, if, else, while, return, void, float.